

REAL ESTATE BOOKING PLATFORM

DATABASE MANAGEMENT SYSTEM MINI-PROJECT REPORT

Submitted by

Student Name

Department of CSBS, BMSIT&M
2025-26

Table of Contents

Abstract	i
List of Figures	ii
List of Tables	iii

Chapter No.	Chapter	Page No.
Chapter 1	Introduction 1.1 Overview of the Project 1.2 Problem Statement 1.3 Objectives of the Project 1.4 Scope of the Project 1.5 Existing System 1.6 Proposed System 1.7 Advantages of Proposed System 1.8 Organization of the Report	1
Chapter 2	System Requirements and Specifications 2.1 Hardware Requirements 2.2 Software Requirements 2.3 Functional Requirements 2.4 Non-Functional Requirements 2.5 Feasibility Study	3
Chapter 3	System Design 3.1 Architecture Diagram 3.2 Data Flow Diagram (DFD) 3.3 ER Diagram 3.4 Schema Diagram 3.5 Normalization 3.6 Data Dictionary 3.7 Table Design	5

Chapter 4	Implementation 4.1 Front-End Design 4.2 Back-End Design 4.3 SQL Queries Used 4.4 Stored Procedures / Triggers (if any) 4.5 Sample Screenshots 4.6 Module Description	9
Chapter 5	Testing 5.1 Testing Objectives 5.2 Test Cases 5.3 Unit Testing 5.4 Integration Testing 5.5 System Testing 5.6 Test Results	13
Chapter 6	Results and Discussion 6.1 Output Screens 6.2 Performance Analysis 6.3 Advantages of the System 6.4 Limitations of the System 6.5 Future Enhancements	16
	WK and SDG Mapping	18
	References	19

Abstract

The Real Estate Booking Platform is a comprehensive database management system designed to streamline the process of property listing, searching, and booking. The system serves three main user roles: Customers, Agents, and Administrators. By leveraging a robust PostgreSQL backend, an Express.js API, and a React frontend, the platform ensures data integrity, real-time availability checking, and secure transactional operations. Key database features such as triggers, stored procedures, and complex aggregations are implemented to enforce business logic directly at the database layer, ensuring optimal performance and consistency.

Chapter 1: Introduction

1.1 Overview of the Project

The project is a web-based Real Estate Booking Platform that connects property agents with potential customers. It provides a centralized database for managing property details, user roles, bookings, payments, and reviews.

1.2 Problem Statement

Traditional real estate processes heavily rely on manual paperwork and disjointed communication channels. There is a lack of a unified system that securely handles property bookings, avoids overlapping reservations, and provides analytical insights for administrators.

1.3 Objectives of the Project

- To develop a relational database system that efficiently handles property and user data.
- To implement strict constraints and triggers preventing duplicate or overlapping bookings.
- To provide role-based access control for Customers, Agents, and Admins.
- To demonstrate advanced SQL operations such as aggregations, joins, and transactions.

1.4 Scope of the Project

The scope includes the development of a full-stack prototype covering user authentication, property CRUD operations, booking and payment flows, and an administrative dashboard for system-wide analytics.

1.5 Existing System

Current manual systems or basic listing websites lack integrated booking and payment processing. They often require offline follow-ups and lack real-time synchronization, leading to double-booking errors.

1.6 Proposed System

The proposed system integrates the database layer directly with the application logic using automated triggers and procedures, ensuring that once a property is booked, its availability is instantly updated globally without manual intervention.

1.7 Advantages of Proposed System

High data integrity, real-time updates, secure transaction processing through PostgreSQL functions, and detailed administrative oversight.

1.8 Organization of the Report

This report is organized into six chapters covering Introduction, System Requirements, System Design, Implementation, Testing, and Results & Discussion.

Chapter 2: System Requirements and Specifications

2.1 Hardware Requirements

- Processor: Intel Core i5 or equivalent
- RAM: 8 GB minimum
- Storage: 50 GB free disk space

2.2 Software Requirements

- Operating System: Windows, macOS, or Linux
- Database: PostgreSQL 16+
- Backend: Node.js, Express.js
- Frontend: React (Vite), Bootstrap 5

2.3 Functional Requirements

The system must allow users to register and log in. Agents must be able to list and edit properties. Customers must be able to browse properties, book them, and make simulated payments. Admins must be able to view all bookings and manage platform-wide properties.

2.4 Non-Functional Requirements

The system must respond within 2 seconds for search queries. Passwords must be securely hashed using bcrypt. The database must maintain ACID properties for all booking transactions.

2.5 Feasibility Study

The project is technically feasible as it utilizes open-source, widely supported web technologies. It is economically feasible due to the lack of licensing costs for PostgreSQL and Node.js.

Chapter 3: System Design

3.1 Architecture Diagram

The system follows a standard 3-tier architecture:

- **Presentation Tier:** React SPA running in the client browser.
- **Application Tier:** Node.js/Express REST API serving as the backend logic.
- **Data Tier:** PostgreSQL database handling persistent storage and business logic triggers.

3.2 Data Flow Diagram (DFD)

Level 0 Context Diagram: The User (Customer/Agent/Admin) interacts with the Booking System, which in turn queries and updates the Database.

Level 1 DFD: Processes include Authentication, Property Management, Booking Processing, and Analytics Generation. Data flows from User Inputs -> Validation -> Database Storage -> Output Dashboards.

3.3 ER Diagram

Entities identified: Users, Agents, Properties, Bookings, Payments, Reviews, Favorites.

Relationships: - A User can have many Bookings (1:N).

- A Booking has one Payment (1:1).

- An Agent manages many Properties (1:N).

- A Property can have many Bookings (1:N).

3.4 Schema Diagram

The relational schema maps the ER diagram into strict tables with Foreign Key constraints enforcing referential integrity. ON DELETE CASCADE is utilized to maintain database cleanliness.

3.5 Normalization

All tables are normalized up to the Third Normal Form (3NF). There are no repeating groups (1NF), all non-key attributes are fully functionally dependent on the primary key (2NF), and there are no transitive dependencies (3NF).

3.6 Data Dictionary

Table Name	Description	Primary Key
users	Stores user credentials and roles	id
agents	Stores agent specific details	id
properties	Stores property listings	id
bookings	Stores booking schedules	id
payments	Stores transaction details	id
reviews	Stores user feedback on bookings	id

3.7 Table Design

Table definitions include strict CHECK constraints (e.g., price > 0, rating BETWEEN 1 AND 5) and default timestamps for audit tracking.

Chapter 4: Implementation

4.1 Front-End Design

The frontend is implemented using React and Vite. Bootstrap 5 is used for responsive layouts. State management is handled via React Context API, specifically for Authentication tracking.

4.2 Back-End Design

The backend uses Express.js. The `pg` library is utilized to connect to PostgreSQL. Parameterized queries are strictly used to prevent SQL injection attacks. Middleware handles JWT validation and role-checking.

4.3 SQL Queries Used

Complex queries including JOINS and Aggregations are used heavily in the Admin Dashboard. For example:

```
SELECT p.city, COUNT(b.id) as booking_count
FROM properties p
JOIN bookings b ON p.id = b.property_id
GROUP BY p.city
ORDER BY booking_count DESC LIMIT 1;
```

4.4 Stored Procedures / Triggers

Trigger 1 (trg_booking_status_change): Automatically updates a property's status to 'booked' when a booking is confirmed.

Trigger 2 (trg_prevent_duplicate_booking): Raises an exception if a booking is attempted on dates that conflict with an active booking.

Function (confirm_booking): Wraps the insertion of a payment record and the updating of a

booking status inside a single atomic database transaction.

4.5 Sample Screenshots

(In a printed report, screenshots of the Home Page, Agent Dashboard, and Admin Booking View would be inserted here.)

4.6 Module Description

- **Auth Module:** Handles bcrypt hashing and JWT issuance.
- **Property Module:** Manages listing, filtering by city/type, and CRUD operations.
- **Booking/Payment Module:** Orchestrates the reservation logic and executes PostgreSQL functions.

Chapter 5: Testing

5.1 Testing Objectives

To verify that all modules function according to the specifications, ensure database constraints prevent invalid data entry, and confirm role-based access security.

5.2 Test Cases

Test Case ID	Description	Expected Result	Status
TC01	Register with existing email	Throw unique constraint error	Pass
TC02	Book a property already booked	Trigger raises Exception	Pass
TC03	Customer accesses Admin route	403 Forbidden Access	Pass
TC04	Search properties by city	Return matching rows via Index	Pass

5.3 Unit Testing

Individual API endpoints were tested using tools like Postman to ensure appropriate JSON responses and status codes are returned for both valid and invalid inputs.

5.4 Integration Testing

Ensured the frontend React components correctly pass JWT tokens in the Authorization header to the backend, and that the backend properly validates them.

5.5 System Testing

The entire booking flow (Login -> Browse -> Book -> Pay -> Status Update) was tested end-to-end to verify that the PostgreSQL functions and triggers execute exactly as intended.

5.6 Test Results

The system successfully passed all test cases. Database integrity was maintained throughout edge-case simulations.

Chapter 6: Results and Discussion

6.1 Output Screens

The final application presents a clean, user-friendly interface. The Admin Dashboard successfully consolidates complex database tables into readable statistics and management tables.

6.2 Performance Analysis

By implementing Database Indexes on `properties(city)` and `bookings(status)`, search filtering and dashboard aggregations perform efficiently even as the dataset grows.

6.3 Advantages of the System

The utilization of database-level triggers and functions drastically reduces the amount of code needed in the Node.js backend, centralizing business logic where data lives, thus guaranteeing consistency regardless of the client.

6.4 Limitations of the System

The current prototype utilizes a mocked payment system rather than integrating with a real-world gateway like Stripe.

6.5 Future Enhancements

Future development could include location-based mapping APIs, real email notifications for booking confirmations, and advanced date-range overlap calculations for properties.

WK (Working Knowledge) Applied

The project successfully demonstrates working knowledge of Relational Database Management Systems (RDBMS), specifically PostgreSQL. Concepts applied include Normalization, Entity-Relationship Modeling, PL/pgSQL Functions, Triggers, and Constraints.

SDG (Sustainable Development Goals) Mapping

- **Goal 9 (Industry, Innovation and Infrastructure):** Digitizing the real estate sector promotes technological innovation and improves transactional infrastructure.
- **Goal 11 (Sustainable Cities and Communities):** By efficiently connecting individuals with housing and commercial spaces, the platform contributes to organized urban resource management.

References

- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. Retrieved from <https://www.postgresql.org/docs/>
- React Documentation. (2024). Meta Open Source. Retrieved from <https://react.dev/>
- Express.js Node.js Web Application Framework. Retrieved from <https://expressjs.com/>
- Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems. Pearson.